Векторни представяния (Embeddings)

Разлика между пълнотекстово и семантично търсене

Богомил Стоянов, Михаил Доброславски

Лекция 5

Приложно AI програмиране

© 2026 Богомил Стоянов, Михаил Доброславски · CC BY-NC-ND 4.0

От Тема 4 към днес

- Досега: моделът чете *текст* и връща *текст*. Днес: как да представим смисъла на текст като **числа**, за да го търсим и сравняваме.
- **Embedding** = вектор с фиксирана размерност, който кодира смисъла; близки вектори ≈ близък смисъл.
- Математиката: **косинусово сходство** (изпитен въпрос №5), с дефиниция, геометрия и връзка със скаларното произведение и евклидовото разстояние.

Защо е важно

Embeddings са основата на *семантичното търсене*, а то стои в основата на векторните бази (Тема 6) и RAG (Теми 7–8). Днес започваме с математиката зад тях.

Дневен ред: Лекция 5

Какво е embedding

Пълнотекстово и семантично търсене

Косинусово сходство

Сходство ≠ релевантност

Код и инженерни въпроси

Чести грешки

Обобщение

Дискусия

Какво е embedding

Какво е embedding

- Вектор от числа с плаваща запетая с **фиксирана размерност** d (напр. 1536), който представя парче текст като точка/посока в d-мерно пространство.
- **Дистрибутивна хипотеза**: думи в сходен контекст имат сходен смисъл; моделът научава това от огромно количество текст. „Куче“ и „кученце“ сочат в близки посоки, а „куче“ и „данъчна декларация“ – в далечни.
- Това е компресия на смисъла *със загуба*, но полезна: пазят се не точните думи, а *семантичните отношения* между тях.

Не е токенизация и не е хеш

Токените са дискретни идентификатори, а embedding е **плътен (dense)** вектор, получен от обучен модел. И не е хеш: *близки* текстове дават *близки* (не еднакви) вектори, т.е. малка промяна в текста → малка промяна във вектора.

Embedding модели и размерност

- Embedding моделът е **отделен** от chat/completion модела; изходът му е вектор, не текст. Напр. OpenAI `text-embedding-3-small` (1536), `text-embedding-3-large` (3072).
- Размерността е **свойство на модела**, не на текста: и една дума, и цял абзац дават вектор с една и съща дължина d.
- **Matryoshka** (параметър `dimensions`): моделите v3 позволяват съкращаване на вектора (напр. 3-large до 256 или 1024), с което пестим памет и време за сметка на малко точност.
- Максимален вход: при OpenAI v3 ∼8192 токена на *един* текст (не на цялата заявка); другите модели имат свои граници.

Едно пространство

Всеки модел има **собствено** пространство, затова векторите от два различни модела не са сравними. Векторизирай (embed) целия текст с *един и същ* модел.

Пълнотекстово и семантично търсене

Два начина да търсим

Пълнотекстово (lexical)

- Съвпадение по *думи*; BM25 (наследник на TF-IDF) подрежда по честота на термина, IDF и дължина.
- Документът е **разреден (sparse)** вектор: измерение за всяка дума от речника, почти само нули.
- Силни страни: точни съвпадения, имена, кодове; евтино, без модел.
- Слаби страни: „кола“ не намира „автомобил“ (vocabulary mismatch).

Семантично

- Векторизираме заявката и документите; връщаме най-близките документи.
- Векторите са **плътни (dense)**: стотици до хиляди ненулеви измерения.
- Силни страни: синоними, парафрази, междуезикова (cross-lingual) близост.
- Слаби страни: пропуска точни кодове и редки думи; може да открие мнима близост; по-скъпо.

Извод

Двата подхода **се допълват** ⇒ хибридно търсене (детайли в Тема 8).

Косинусово сходство

Косинусово сходство: дефиниция (изпитен въпрос №5)

За два вектора $u, v \in \mathbb{R}^d$:

$$\text{cos_sim}(u, v) = \frac{u \cdot v}{\|u\| \, \|v\|} = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_{i=1}^{d} u_i^2} \, \sqrt{\sum_{i=1}^{d} v_i^2}}$$

- **Скаларно произведение** $u \cdot v = u_1 v_1 + u_2 v_2 + \cdots + u_d v_d$.
- **Норма** $\|u\| = \sqrt{u_1^2 + \cdots + u_d^2}$ (евклидова дължина).
- Стойности в $[-1, 1]$: 1 = еднаква посока (максимално сходни), 0 = ортогонални (несвързани), -1 = противоположни посоки.

Пример на ръка

Нека $u = (1, 2)$ и $v = (2, 1)$. Тогава $u \cdot v = 1 \cdot 2 + 2 \cdot 1 = 4$ и $\|u\| = \|v\| = \sqrt{1^2 + 2^2} = \sqrt{5}$, следователно

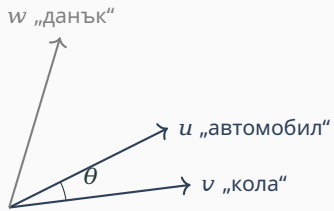
$$\text{cos_sim}(u, v) = \frac{4}{\sqrt{5}\sqrt{5}} = 0{,}8.$$

- $w = (-2, 1)$: $u \cdot w = -2 + 2 = 0 \Rightarrow \text{cos_sim}(u, w) = 0$ (ортогонални).
- $-u = (-1, -2)$: $\text{cos_sim}(u, -u) = -5/5 = -1$ (противоположни посоки).
- $2u = (2, 4)$: $\text{cos_sim}(u, 2u) = 1$; дължината не влияе.

А при реални embeddings?

Стойностите обикновено са в тесен **положителен** диапазон; -1 *не* означава „противоположен смисъл“. Праговете зависят от модела, затова сравнявай **подреждането**, а не абсолютната стойност.

Геометрична интуиция: косинус на ъгъла



- cos_sim е косинусът на ъгъла θ.
- Малък ъгъл → близо до 1; прав ъгъл → 0.
- „кола“ и „автомобил“ сочат почти в една и съща посока, а „данък“ – в доста различна.

Защо косинус, а не евклидово разстояние

Косинусът мери **посока**, не дължина. При векторите от броене на думи (TF-IDF) дълъг и кратък документ по една тема са далеч, но сочат сходно. Embedding моделите обикновено връщат нормализирани вектори и там двете мерки подреждат еднакво (следващият слайд).

Нормализирани вектори и евклидово разстояние

- **Единичен вектор** (unit vector): $\|u\| = 1$. Ако $\|u\| = \|v\| = 1$, знаменателят е 1 и

$$\text{cos_sim}(u, v) = u \cdot v.$$

- Много embedding модели (вкл. OpenAI) връщат **вече нормализирани** вектори ⇒ косинусовото сходство е просто скаларно произведение (по-евтино за изчисление).
- **Връзка с евклидовото разстояние**: за нормализирани вектори

$$\|u - v\|^2 = 2 - 2\,(u \cdot v),$$

  т.е. подреждането по косинус и по евклидово разстояние е *едно и също* (монотонна връзка).
- **Косинусово разстояние** $= 1 -$ cos_sim (превръща сходство в „разстояние“).

Косинусово сходство „на ръка“ (как работи отвътре)

Първо го пресмятаме сами, а после съзнателно използваме готова помощна функция (helper).

```
import math

def dot(a, b):   return sum(x * y for x, y in zip(a, b))   # a·b по координати
def norm(v):     return math.sqrt(sum(x * x for x in v))   # Питагор в d измерения

def cos_sim(a, b):
    return dot(a, b) / (norm(a) * norm(b))

# Ако векторите вече са нормализирани: norm() е 1 и cos_sim се свежда до dot.
# Проверка: norm(vector) ~= 1.0
```

Векторът е просто списък от числа; не са нужни никакви библиотеки. За нормализирани вектори знаменателят е 1, тоест `cos_sim` = `dot`, точно както следва от формулата.

Сходство ≠ релевантност

Сходство ≠ релевантност (важно уточнение)

- Високото косинусово сходство означава близост в пространството на embeddings, но **не** гарантира, че резултатът е полезен за конкретната задача.
- Примери за разминаване:
  - **антоними**/противоположни твърдения често са близки (една тема, обратен смисъл);
  - стилистична/тематична близост без фактическо съответствие;
  - чувствителност към формулировката на заявката.

Практически извод

Сходството е силен сигнал при подбора на кандидати (*candidate generation*), но окончателната преценка за релевантност често изисква повторно класиране (**re-ranking**, Тема 8).

Код и инженерни въпроси

Пакетно генериране на embeddings (batching)

```python
from openai import OpenAI
client = OpenAI()

texts = ["куче", "кученце", "автомобил", "данъчна декларация"]
resp = client.embeddings.create(
    model="text-embedding-3-small",   # 1536 измерения
    input=texts,                       # batch: много текстове с ЕДНА заявка
)
vectors = [d.embedding for d in resp.data]
# len(vectors[0]) == 1536
```

Пакетната обработка (**batching**) е много по-бърза от отделна заявка за всеки текст и пести лимита на заявки (rate limit). Embeddings се таксуват по токени и обикновено струват много по-малко от генерирането на текст (провери актуалните цени в ценоразписа на доставчика).

Мини семантично търсене (без БД)

```python
def search(query, docs, doc_vecs, k=3):
    q = client.embeddings.create(
        model="text-embedding-3-small", input=[query]).data[0].embedding
    scores = sorted(
        ((cos_sim(q, v), d) for d, v in zip(docs, doc_vecs)),
        reverse=True,
    )
    return scores[:k]   # top-k по косинусово сходство
```

Ако заявката е „автомобил“, а документът съдържа само „кола“, пълнотекстовото търсене не намира съвпадение (0), а семантичното открива близост. Това е нагледен пример за vocabulary mismatch.

Същото с локален embedding модел (Ollama)

Embeddings могат да се генерират и локално, без API ключ:

```python
import ollama

resp = ollama.embed(                          # изисква: ollama pull bge-m3
    model="bge-m3",
    input=["куче", "кученце", "автомобил"],
)
vectors = resp["embeddings"]                  # списък от вектори (тук 1024 измерения)
```

Друг модел ⇒ друго пространство и друга размерност (1024, не 1536), затова не смесвай вектори от различни модели. Самата `cos_sim` се прилага по същия начин.

Вариант на TypeScript (готовата помощна функция)

```ts
import { openai } from "@ai-sdk/openai";
import { embedMany, cosineSimilarity } from "ai";

const { embeddings } = await embedMany({
  model: openai.textEmbeddingModel("text-embedding-3-small"),
  values: ["куче", "кученце", "автомобил"],
});

cosineSimilarity(embeddings[0], embeddings[1]); // число в [-1, 1]
```

`cosineSimilarity` е готовата версия на нашия `cos_sim` и вече знаем какво изчислява вътрешно.

Инженерни въпроси: цена, латентност, памет

- **Batching + кеширане**: един и същ текст дава един и същ вектор → кеширай embeddings, за да не плащаш повторно.
- **Латентност/rate limits**: при индексиране на голям обем текст планирай паралелна обработка и повторни опити (retry).
- **Съхранение**: d числа с плаваща запетая на документ (напр. 1536×4 байта ≈ 6 KB). При милиони документи паметта расте с размерността.
- **float32** обикновено стига, така че не пази float64 без нужда.

Мотивация за Тема 6

Линейното сравняване с всеки вектор става твърде бавно при милиони документи ⇒ нужни са **векторни бази** с индекси за приблизително търсене на най-близки съседи (ANN).

Чести грешки

Чести грешки и важни уточнения

- **Сравняване на вектори от различни модели** (или различна размерност): безсмислено, защото пространствата са различни.
- **Забравена нормализация** при пряко скаларно произведение върху ненормализирани вектори → изкривени сходства. Нормализирай или ползвай пълната формула.
- **Бъркане на сходство с релевантност**: антонимите и противоположностите често са близки.
- **Отделна заявка за всеки текст** вместо пакетна обработка: бавно и хаби лимита на заявки.
- **Пренебрегване на максималния вход** (~8192 токена): дългият текст се разделя на парчета (chunking, Теми 7–8) преди векторизиране.
- **Очакване, че семантичното търсене замества пълнотекстовото**: то се проваля при кодове, идентификатори и редки термини, затова е нужно хибридно търсене.
- **float64 без нужда**: удвоява паметта при милиони вектори.

Обобщение

Какво научихме днес

- **Embedding** = плътен вектор с фиксирана размерност; близост ≈ смисъл. Отделен от chat модела; едно пространство за целия текст.
- **Косинусово сходство** $= \frac{u \cdot v}{\|u\|\|v\|} \in [-1, 1]$; косинус на ъгъла; за нормализирани вектори = скаларно произведение.
- **Пълнотекстово (BM25) и семантично**: точни думи или смисъл; двата подхода се допълват (хибридно търсене, Тема 8).
- **Сходство ≠ релевантност** → re-ranking (Тема 8).
- Инженерни въпроси: batching, кеширане, float32; линейното търсене е твърде бавно при милиони вектори → векторни бази (Тема 6).

Следваща лекция

Тема 6 е за векторните бази данни: ефективно съхранение и ANN индекси за милиони вектори.

Връзка с изпитния конспект

- **Въпрос 5**: „Математическа същност на векторните представяния (Embeddings) и косинусово сходство.“ → покрит изцяло:
  - *Математическа същност*: вектор с фиксирана размерност, embedding модели, размерност, плътни и разредени (dense/sparse) вектори.
  - *Косинусово сходство*: формула (скаларно произведение, разделено на произведението от нормите), стойности $[-1, 1]$, косинус на ъгъла, посока, а не дължина, нормализирани вектори (= скаларно произведение), връзка с евклидовото разстояние.
  - *Отвъд дефиницията*: сходство ≠ релевантност.

Дискусия

Теми

1. Кога пълнотекстовото търсене превъзхожда семантичното? Дай пример за заявка, при която семантичното търсене се проваля.
2. Косинусово сходство ≠ смислова релевантност: къде се разминават?
3. Какви embeddings ползваме (модел, език, домейн) и как изборът променя резултатите?

Библиография

- Alammar & Grootendorst (2024). *Hands-On Large Language Models* (token & sentence embeddings, семантично търсене).
- OpenAI, Embeddings guide: `https://developers.openai.com/api/docs/guides/embeddings`
- Vercel AI SDK, `embed` / `embedMany` / `cosineSimilarity`: `https://ai-sdk.dev/docs`
- Gemini API, Embeddings: `https://ai.google.dev/gemini-api/docs/embeddings`
- Robertson & Zaragoza (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*.